Open the application.yml file for your Catalog Service project and apply the following configuration to make the interaction with Config Service more resilient. Once again, there's no universal rule for setting up timeouts. Depending on your architecture and infrastructure characteristics, you might need to adjust these values.

Listing 4.16 Making Spring Cloud Config Client more resilient

```
spring:
  application:
    name: catalog-service
  config:
    import: "optional:configserver:"
  cloud:
    config:
      uri: http://localhost:8888
      request-connect-timeout: 5000      ❶
      request-read-timeout: 5000         ❷
```

❶ Timeout on waiting to connect to the config server (ms)

❷ Timeout on waiting to read configuration data from the config server (ms)

Even if Config Service is replicated, there's still a chance it will be temporarily unavailable when a client application like Catalog Service starts up. In that scenario, you can leverage the *retry* pattern and configure the application to try again to connect with the config server before giving up and failing. The retry implementation for Spring Cloud Config Client is based on Spring Retry, so you'll need to add a new dependency to the build.gradle file for your Catalog Service project. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 4.17 Adding dependency for Spring Retry in Catalog Service

```
dependencies {
  ...
  implementation 'org.springframework.retry:spring-retry'
}
```